

ДЛЯ MANUAL QA / JUNIOR / НОВИЧКОВ

AI QA

без паники:

терминал, Python, pytest и AI

Короткий вводный гайд, чтобы спокойно начать основной AI QA Practice Kit.

ЧТО СТАНЕТ ПОНЯТНО

- 01 Какой маршрут выбрать
- 02 Как не бояться терминала
- 03 Что такое venv, pytest и assert
- 04 Как просить AI о помощи и читать ошибки

START

AI QA Beginner Guide

Этот гайд нужен, чтобы спокойно войти в основной AI QA Practice Kit.

Если ты manual QA и видишь слова pytest, RAGAS, Locust, DeepEval, MCP - нормально чувствовать перегруз. Тебе не нужно понимать все сразу. Сначала нужно научиться двигаться по шагам.

Что важно

Основной workbook - это не книга "прочитал и забыл". Это практический проект. Там будут команды, файлы, тесты, dataset, метрики и отчеты.

Этот beginner guide помогает подготовиться:

- понять, что именно ты будешь делать;
- не бояться терминала;
- научиться запускать базовые команды;
- правильно просить AI о помощи;
- фиксировать ошибки;
- выбрать свой маршрут.

Чего этот гайд не требует

Тебе не нужно:

- быть программистом;
- знать ML;
- писать сложную автоматизацию;
- понимать RAGAS с первой страницы;
- проходить все идеально.



Какой результат нужен

После этого гайда у тебя должны быть:

- выбранный маршрут прохождения;
- понимание базовых терминов;
- рабочая папка проекта;
- шаблон разбора ошибки;
- готовность открыть основной workbook без паники.

Главная мысль

Ты не обязан стать AQA за неделю. Твоя цель - начать использовать AI как рабочий инструмент QA и постепенно собрать портфолио-кейс.

Практика

Создай файл MY_AI_QA_START.md и ответь:

1. Кто я сейчас: manual, junior, AQA, lead?
2. Чего я боюсь в технической части?
3. Что хочу получить через 30 дней?
4. Сколько времени в неделю реально готов уделять?

START

Три маршрута прохождения

Люди приходят в AI QA с разным опытом. Поэтому не надо всем идти одинаково.

Выбери маршрут и не сравнивай себя с теми, кто уже пишет автотесты.

Маршрут 1: Manual QA

Твоя цель: понять AI QA workflow и собрать артефакты, даже если код пока дается тяжело.

Фокус:

- risk map;
- prompts;
- базовые API-тесты;
- golden dataset;
- понимание RAG-метрик;
- release report.

Можно временно пропустить:

- сложный CI;
- глубокий Playwright;
- кастомный код метрик;
- оптимизацию Locust.

Маршрут 2: AQA / SDET

Твоя цель: встроить AI в automation workflow и добавить AI evaluation.

Фокус:

- pytest структура;



- API tests;
- Playwright;
- eval scripts;
- custom metrics;
- CI;
- load profile.

Маршрут 3: QA Lead / Middle+

Твоя цель: понять, как управлять качеством AI-фичи на уровне процесса.

Фокус:

- risk map;
- golden dataset strategy;
- thresholds;
- security findings;
- release gate;
- отчет для команды.

Код важен, но не главный. Главный результат - понятный quality process.

Как понять, что маршрут выбран правильно

Если ты думаешь	Выбирай
"Я боюсь терминала"	Manual QA
"Я хочу писать тесты быстрее"	AQA
"Мне надо управлять качеством AI-фич"	Lead
"Я новичок и хочу портфолио"	Manual QA, потом AQA

Практика

В MY_AI_QA_START.md добавь:



Мой маршрут:
Мой главный фокус:
Что я могу пропустить на первом проходе:
Что обязательно должен сделать:

START

Терминал без паники

Терминал - это место, где ты запускаешь команды. Он не обязан быть страшным.

В основном workbook команды нужны для:

- создания окружения;
- установки пакетов;
- запуска тестов;
- сборки PDF/отчетов;
- запуска Locust или Playwright.

Главное правило

Всегда смотри, из какой папки запускаешь команду.

В cmd команда:

```
cd
```

покажет текущую папку.

Как понять, что ты в нужной папке

Если проект лежит в PDF_guid, ты должен видеть примерно:

```
C:\Users\...\PDF_guid>
```

Если ты запускаешь команду не из той папки, Python может не найти файлы.

Команды, которые нужно знать

```
dir
```

Показать файлы в папке.

```
cd folder_name
```

Зайти в папку.

```
cd ..
```

Выйти на уровень выше.

```
python --version
```

Проверить Python.

Частая ошибка

Ты копируешь команду:

```
python scripts\build_pdf.py
```

Но находишься не в корне проекта. Тогда scripts\build_pdf.py не найден.

Решение: сначала перейти в корень проекта.

Как просить AI помочь

Плохо:

```
Не работает команда.
```

Хорошо:

```
Я в Windows cmd.  
Текущая папка: [вставь путь из cd].  
Команда: [вставь команду].  
Ошибка: [вставь полный текст].  
Объясни простыми словами, что проверить.
```

Практика



Открой cmd и выполни:

```
cd  
dir  
python --version
```

Запиши результат в SETUP_NOTES.md.

START

Python и venv без боли

Python нужен, чтобы запускать тесты и скрипты. venv нужен, чтобы зависимости проекта не смешивались со всей системой.

Представь venv как отдельную коробку для инструментов конкретного проекта.

Почему нельзя ставить все глобально

Если ставить пакеты глобально:

- разные проекты могут конфликтовать;
- сложно понять, откуда ошибка;
- один пакет обновился и сломал другой проект;
- перенос проекта становится болью.

Базовые команды для CMD

Создать окружение:

```
python -m venv .venv
```

Активировать:

```
call .venv\Scripts\activate.bat
```

После активации ты увидишь:

```
(.venv) C:\...\PDF_guid>
```

Поставить зависимости:

```
python -m pip install -r requirements.txt
```

Как понять, что venv активирован

В начале строки есть (.venv).

Если его нет, активируй снова:

```
call .venv\Scripts\activate.bat
```

Частые проблемы

Проблема	Что проверить
python не найден	установлен ли Python и добавлен ли в PATH
activate открывается как файл	ты в PowerShell или cmd? для cmd нужен activate.bat
pip ругается	скопируй полный текст ошибки
пакет не ставится	проверь интернет и версию Python

Практика

Создай SETUP_NOTES.md:

```
# Setup notes

OS:
Python version:
Venv created: yes/no
Venv activated: yes/no
Installed packages:
Errors:
How fixed:
```

START

Pytest на пальцах

Pytest - это инструмент для запуска тестов на Python.

Тест - это маленькая функция, которая проверяет ожидание.

Самый простой тест

```
def test_sum():  
    assert 2 + 2 == 4
```

assert значит: "я ожидаю, что это правда".

Если правда - тест прошел. Если нет - тест упал.

Пример ближе к QA

```
def test_response_has_answer():  
    response = {  
        "answer": "Возврат возможен в течение 14 дней.",  
        "citations": ["refund_policy.md"]  
    }  
  
    assert "answer" in response  
    assert response["citations"]
```

Мы проверяем:

- поле answer есть;
- список citations не пустой.

Почему это важно для AI QA

AI-ответ может быть разным, но контракт должен быть стабильным.

Не надо сразу проверять весь смысл ответа. Сначала проверь простое:

- ответ есть;
- источник есть;
- тип данных правильный;
- no-data сценарий не выдумывает.

Как запускать pytest

```
python -m pytest
```

Если хочешь запустить конкретный файл:

```
python -m pytest tests\functional\test_chat_contract.py
```

Как читать результат

```
1 passed
```

Все хорошо.

```
1 failed
```

Тест упал. Читай ниже, какой assert не прошел.

Практика

Создай файл first_test.py:

```
def test_ai_response_has_source():
    response = {
        "answer": "В Full входит PDF и roadmap.",
        "citations": ["pricing.md"]
    }

    assert response["answer"]
    assert response["citations"]
```

Запусти:

```
python -m pytest first_test.py
```

START

Как просить AI о помощи

AI помогает лучше, если ты даешь ему контекст.

Не надо писать "не работает". Надо писать как QA: что делал, что ожидал, что получил.

Формула хорошего запроса

```
Контекст:  
Что я делаю.  
  
Команда или код:  
Что запустил.  
  
Ожидал:  
Что должно было быть.  
  
Получил:  
Какая ошибка.  
  
Ограничения:  
Не предлагай опасные действия, не удаляй проект, объясняй простыми словами.
```

Пример

```
Я прохожу AI QA workbook на Windows в cmd.  
Пытаюсь активировать venv.  
  
Команда:  
call .venv\Scripts\activate.bat  
  
Ожидал:  
Увидеть (.venv) в начале строки.  
  
Получил:  
[вставить ошибку]  
  
Объясни простыми словами, что проверить.
```

Prompt для объяснения кода

Объясни этот pytest тест как для manual QA.
Сначала скажи, какой риск он закрывает.
Потом объясни каждую строку.
В конце скажи, как его можно улучшить.

Prompt для ошибки

Я новичок в Python.
Вот команда, которую я запустил:
[команда]

Вот полный текст ошибки:
[ошибка]

Объясни:

1. что означает ошибка;
2. что проверить первым;
3. какие 2 безопасных шага попробовать.

Что не надо делать

- Не вставляй секреты.
- Не вставляй реальные токены.
- Не проси AI "почистить все".
- Не запускай команды, которые не понимаешь.
- Не соглашайся на удаление папок без причины.

Практика

Сохрани 3 prompts:

- объяснить ошибку;
- объяснить тест;
- попросить AI сделать review твоего решения.

START

Как читать ошибки

Ошибка - это не провал. Это данные.

QA уже умеет работать с дефектами. Ошибка в терминале - такой же дефект: есть шаги, expected, actual и окружение.

Что фиксировать

```
ОС:  
Папка:  
Команда:  
Ожидал:  
Получил:  
Полный текст ошибки:  
Что уже пробовал:
```

Пример error report

```
# Error report  
  
OS: Windows 11  
Shell: cmd  
Folder: C:\...\PDF_guid  
Command: python -m pytest  
  
Expected:  
Tests run.  
  
Actual:  
ModuleNotFoundError: No module named pytest  
  
What I tried:  
Activated venv.  
  
Possible reason:  
pytest not installed in venv.
```

Частые ошибки

Ошибка	Возможная причина
ModuleNotFoundError	пакет не установлен



File not found	команда из неправильной папки
Permission denied	нет прав или файл занят
SyntaxError	ошибка в коде
Connection refused	API/server не запущен

Как не застревать

1. Не переписывай все сразу.
2. Скопируй полный текст ошибки.
3. Проверь папку.
4. Проверь venv.
5. Спроси AI по шаблону.
6. Запиши решение.

Практика

Создай `templates/error-report.md` и используй его каждый раз, когда команда не сработала.

START

Мини-словарь AI QA

AI QA

QA, который умеет использовать AI в работе и проверять AI-фичи.

Prompt

Инструкция для AI. В продукте prompt нужно ревьюить как инженерный артефакт.

Vibe coding

Работа, где AI помогает быстро писать черновики кода. В QA это безопасно только с review, запуском и проверками.

API

Интерфейс, через который системы общаются друг с другом. QA часто проверяет API через requests/pytest.

Pytest

Python-инструмент для запуска тестов.

Playwright

Инструмент для браузерных UI-тестов.

RAG



Retrieval-Augmented Generation. Модель отвечает с использованием найденного контекста из базы знаний.

Golden dataset

Набор эталонных вопросов и ожидаемых признаков ответа для проверки AI-фичи.

Faithfulness

Метрика: подтверждается ли ответ найденным контекстом.

Answer Relevancy

Метрика: отвечает ли ответ на вопрос пользователя.

Context Recall

Метрика: нашел ли retriever все нужные chunks.

Context Precision

Метрика: насколько найденные chunks полезны и хорошо ранжированы.

DeepEval

Инструмент для LLM evaluation в стиле тестов.

RAGAS

Инструмент для оценки RAG pipeline.

DeepTeam



Инструмент/подход для adversarial testing AI-систем.

Locust

Инструмент для нагрузочного тестирования.

Release gate

Набор условий, которые должны быть выполнены перед релизом.



START

Чекпоинт перед основным workbook

Перед тем как идти в AI QA Practice Kit, проверь готовность.

Минимальный чек-лист

Проверка	Готово
Я выбрал маршрут прохождения	
Я понимаю, зачем нужен терминал	
Я умею проверить текущую папку	
Я понимаю, что такое venv	
Я запускал простой pytest тест	
Я знаю, как просить AI об ошибке	
У меня есть error-report template	
Я понимаю базовые слова RAG/dataset/metric	

Если что-то не готово

Это нормально. Вернись к нужной главе и сделай практику.

Основной workbook можно проходить не идеально. Но если ты вообще не запускал команду и не видел pytest, технические главы будут ощущаться тяжелее.

Как идти дальше

1. Открой основной workbook.
2. Прочитай первые 3 главы.
3. Не пропускай артефакты.
4. Если застрял - заполни error report.



5. Не пытайся понять все метрики за один день.

Главный критерий

Ты готов, если можешь сказать:

Я понимаю, что буду собирать AI QA mini-project, и знаю, как действовать, если команда или тест не запустились.